Samuel Piorkowski

# Mutation-rate sensitivity of phylogenomic tree reconstruction using Aevol 4b simulated genomes

Pilot benchmark report for aevol_phylo_mutrate, rep_001

**Abstract**

Phylogenomic methods are used to reconstruct evolutionary relationships from genome-scale sequence data, but evaluating their accuracy can be difficult when working with real organisms. In empirical datasets, the true evolutionary history is usually not known with certainty. Researchers can compare inferred trees to taxonomy, prior studies, or biological expectations, but those comparisons are still indirect. Simulated evolution provides a useful alternative because the researcher can know the "true" tree in advance and then test whether different methods can recover it from the final sequences.

This pilot benchmark used Aevol 4b simulated genomes to test how increasing local sequence-level mutation rate affected recovery of a known 16-tip species tree. Aevol 4b was a good fit for this project because it produces ATGC-encoded genomes that can be analyzed with standard bioinformatics tools while still evolving through an explicit genotype-to-phenotype-to-fitness model. In other words, the genomes are artificial, but they are still sequence-based and compatible with normal FASTA-processing and phylogenomic workflows. The original Aevol 4b work showed that this kind of artificial-life simulation can produce data usable by bioinformatics pipelines, and this project applies that general idea to a smaller, focused mutation-rate stress test.

Five mutation-rate regimes were simulated from a shared Aevol setup: very_low, low, normal, high, and very_high. Only three local mutation parameters were scaled: POINT_MUTATION_RATE, SMALL_INSERTION_RATE, and SMALL_DELETION_RATE. Larger rearrangement-rate parameters were kept at their baseline values so that this first version of the benchmark focused specifically on local sequence-level mutation pressure. Final FASTA files were collected, cleaned, quality checked, and analyzed with three phylogenomic workflows: Mash/BioNJ, MAFFT/IQ-TREE, and Mauve/IQ-TREE. Inferred trees were then compared to the known simulated topology using normalized Robinson-Foulds distance.

In the first completed replicate, rep_001, all successful methods recovered the true topology at very_low and low mutation rates. Mash/BioNJ began to show topological error at normal and degraded further under high and very_high. MAFFT/IQ-TREE performed best overall in

this pilot replicate, recovering the true topology through normal and showing less topological error than Mash/BioNJ at higher mutation rates. Mauve/IQ-TREE was nearly accurate through normal (normalized RF = 0.077), but did not produce scored trees at high or very_high because the strict shared-core alignment rule left most tips represented by gaps at those mutation rates. These pilot results suggest that increasing local mutation pressure reduces tree recoverability in a method-dependent way. More replicates are needed before making strong statistical claims, but rep_001 demonstrates a complete workflow for simulation, data cleaning, phylogenetic inference, tree scoring, and interpretation across a controlled mutation-rate gradient.

---

## 1. Introduction

### 1.1 Why simulated phylogenomics is useful

Phylogenetics is the study of evolutionary relationships. In its simplest form, a phylogenetic tree is a branching diagram that represents how organisms, genes, or sequences are related through common ancestry. The tips of the tree represent the sampled organisms or sequences, while the internal branches represent inferred ancestral relationships. When the input data are genome-scale, rather than a single gene or short sequence, the analysis is often described as phylogenomics.

Phylogenomic inference is important because it gives researchers a way to reconstruct evolutionary history from sequence data. For example, it can help identify which organisms are closely related, how microbial lineages diverged, or whether a set of genes shares the same evolutionary history. However, there is a major challenge when evaluating phylogenetic methods: in real biological datasets, the true tree is usually unknown. We can infer a tree, but we often cannot directly prove that it is the exact historical tree.

That is why simulated datasets are useful. In a simulation, the researcher can define or record the true evolutionary history before generating the final sequences. Those final sequences can then be treated like normal genomic data, but the true answer is still available. This makes it possible to ask a much cleaner benchmarking question: given only the final genomes, can a method reconstruct the tree that actually produced them?

This project uses that logic. The goal was not only to infer trees from Aevol 4b genomes, but to measure how accurately different workflows recovered a known 16-tip tree. Because the tree was known, the project could move beyond “does this tree look plausible?” and ask “how close is this inferred tree to the true simulated topology?”

### 1.2 Why Aevol 4b is a good fit for this project

Aevol is a forward-in-time evolutionary simulator. That means populations evolve generation by generation through processes such as mutation, replication, and selection. In Aevol, mutations are applied to explicit genome sequences, and the consequences of those mutations are evaluated through a genotype-to-phenotype-to-fitness model. A genotype is the genetic sequence, a phenotype is the expressed or functional outcome of that sequence, and fitness describes how successful an organism is at contributing to the next generation. This matters because mutations are not simply assigned predefined effects. Instead, their effects depend on how the changed genome is decoded and selected within the simulation.

Aevol 4b is especially useful for a bioinformatics benchmark because it uses ATGC-encoded genomes. Earlier or standard Aevol models used binary genome representations, which are useful for artificial-life studies but less directly compatible with typical sequence-analysis tools. Aevol 4b bridges that gap by producing genome sequences that can be stored, cleaned, aligned, compared, and analyzed with ordinary bioinformatics tools. The Aevol 4b paper specifically frames this as a bridge between artificial-life simulation and bioinformatics, showing that artificial evolution can produce sequence data that can be processed by off-the-shelf bioinformatic methods.

For this project, Aevol 4b is useful for two reasons. First, it allows controlled simulation under a known evolutionary history. Second, it produces final genomes that can be handled like normal FASTA files. That combination makes it a good system for a portfolio-style bioinformatics benchmark because it requires both evolutionary reasoning and practical computational workflow development.

1.3 Project framing and central question

This project is best framed as a focused pilot benchmark inspired by the Aevol 4b approach. It uses the same general logic of simulated evolution followed by bioinformatics analysis, but it applies that logic to a smaller and more targeted question: how does local sequence-level mutation rate affect tree reconstruction?

The central question was:

How does increasing local sequence-level mutation rate affect the ability of different phylogenomic workflows to recover a known simulated 16-tip species tree from Aevol 4b genomes?

This is a useful question because mutation rate affects phylogenetic inference in more than one way. If mutation rate is too low, there may not be enough differences among sequences to identify relationships confidently. These useful differences are often called phylogenetically informative sites, meaning positions in an alignment that help distinguish one branching pattern from another. If mutation rate is moderate, lineages may accumulate

enough differences to become distinguishable while still preserving recognizable shared sequence. If mutation rate is very high, the opposite problem can occur. Repeated mutations, small insertions and deletions, genome shortening, or alignment difficulty can obscure the original evolutionary signal.

In other words, “more mutation” does not automatically mean “more useful information.” At some point, additional mutation can make genomes harder to compare. This project tests that idea across five mutation-rate regimes and three different tree-building workflows.

### 1.4 Aims and hypotheses

This pilot benchmark had three main aims.

Aim 1: Generate comparable final Aevol 4b genomes across five local mutation-rate regimes.

Aim 2: Infer phylogenetic trees from those final genomes using workflows that rely on different kinds of sequence signal.

Aim 3: Compare each inferred tree to the known 16-tip simulated tree using topology-based scoring.

The main expectation was that mutation rate would have a non-uniform effect on tree recovery. The lower mutation-rate conditions were expected to preserve similarity among genomes, but they could potentially contain fewer informative sites. The moderate condition was expected to provide a useful balance between divergence and comparability. The higher mutation-rate conditions were expected to be more difficult because strong divergence, indel accumulation, genome shortening, or alignment failure could reduce the amount of usable signal.

A second expectation was that the workflows would not fail in the same way. Mash/BioNJ, MAFFT/IQ-TREE, and Mauve/IQ-TREE each process the genome data differently. A distance-based workflow, an alignment-based maximum-likelihood workflow, and a whole-genome core-alignment workflow may respond differently to the same mutation-rate stress.

---

## 2. Materials and Methods

### 2.1 Overview of the computational workflow

The project was organized as a reproducible workflow under the directory aevol_phylo_mutrate. Configuration files, scripts, raw data, cleaned data, simulation runs, phylogenetic outputs, scoring outputs, and plots were separated into dedicated folders. This

was important because the project was not just a single analysis. It was a small end-to-end benchmark pipeline.

The workflow had two major phases. The first phase generated the simulated genomes. This included creating the known tree, generating mutation-rate-specific Aevol parameter files, pre-evolving one shared ancestor, running branch-by-branch simulations across the known tree, collecting final tip FASTA files, and performing FASTA QC. The second phase used those cleaned FASTA files for phylogenetic inference, ran three workflows, scored inferred trees against the known topology, and generated summary plots.

This organization also matters from a skills perspective. A central outcome of the project is not only the biological interpretation of the pilot result, but the construction of a reproducible bioinformatics workflow. The project demonstrates simulation design, file management, scripting, FASTA cleaning, software environment tracking, tree inference, tree validation, and results visualization. The uploaded outline already documents these pieces as part of the rep_001 workflow.

### 2.2 Aevol 4b simulation framework

Simulations were performed using Aevol 4b, the four-base version of Aevol. In this framework, organisms have genomes that are decoded into phenotypes, and those phenotypes are used to compute fitness. Reproduction and mutation occur over generations, so populations evolve forward in time. The 4-Bases version differs from Standard Aevol because it uses ATGC sequence rather than a binary genome representation. This makes its outputs much easier to pass into standard FASTA-based tools.

This point is worth explaining carefully for a non-specialist audience. A FASTA file is a plain-text sequence file format. Each sequence has a header line, usually beginning with >, followed by one or more lines of sequence characters. In this project, the terminal genomes were exported as FASTA sequences, cleaned so that their labels matched the expected tip names, and then used as input for different phylogenetic workflows.

Aevol 4b genomes should still be interpreted as simulated artificial-life genomes. They are not meant to perfectly reproduce a real microbial genome. Their value here is that they provide controlled, sequence-based data with a known evolutionary history. That makes them useful for benchmarking computational methods.

### 2.3 Experimental design

The design used one shared pre-evolved ancestor, one fixed balanced 16-tip tree, five mutation-rate regimes, and one completed replicate, rep_001. A replicate is one full run of

the experimental design. Because only one replicate has been completed so far, this report should be interpreted as a pilot benchmark rather than a final statistical study.

The terminal species were labeled S01 through S16. These terminal species are the tips of the tree, meaning they are the final genomes sampled at the end of the simulated evolutionary process. A balanced tree was used, meaning the branching structure was evenly arranged rather than highly uneven. This made the first version easier to debug and interpret.

The five mutation-rate regimes were:

| Condition | Multiplier | Parameters scaled |
|---|---|---|
| very_low | 0.0625x | POINT_MUTATION_RATE, SMALL_INSERTION_RATE, SMALL_DELETION_RATE |
| low | 0.25x | POINT_MUTATION_RATE, SMALL_INSERTION_RATE, SMALL_DELETION_RATE |
| normal | 1.0x | POINT_MUTATION_RATE, SMALL_INSERTION_RATE, SMALL_DELETION_RATE |
| high | 4.0x | POINT_MUTATION_RATE, SMALL_INSERTION_RATE, SMALL_DELETION_RATE |
| very_high | 16.0x | POINT_MUTATION_RATE, SMALL_INSERTION_RATE, SMALL_DELETION_RATE |

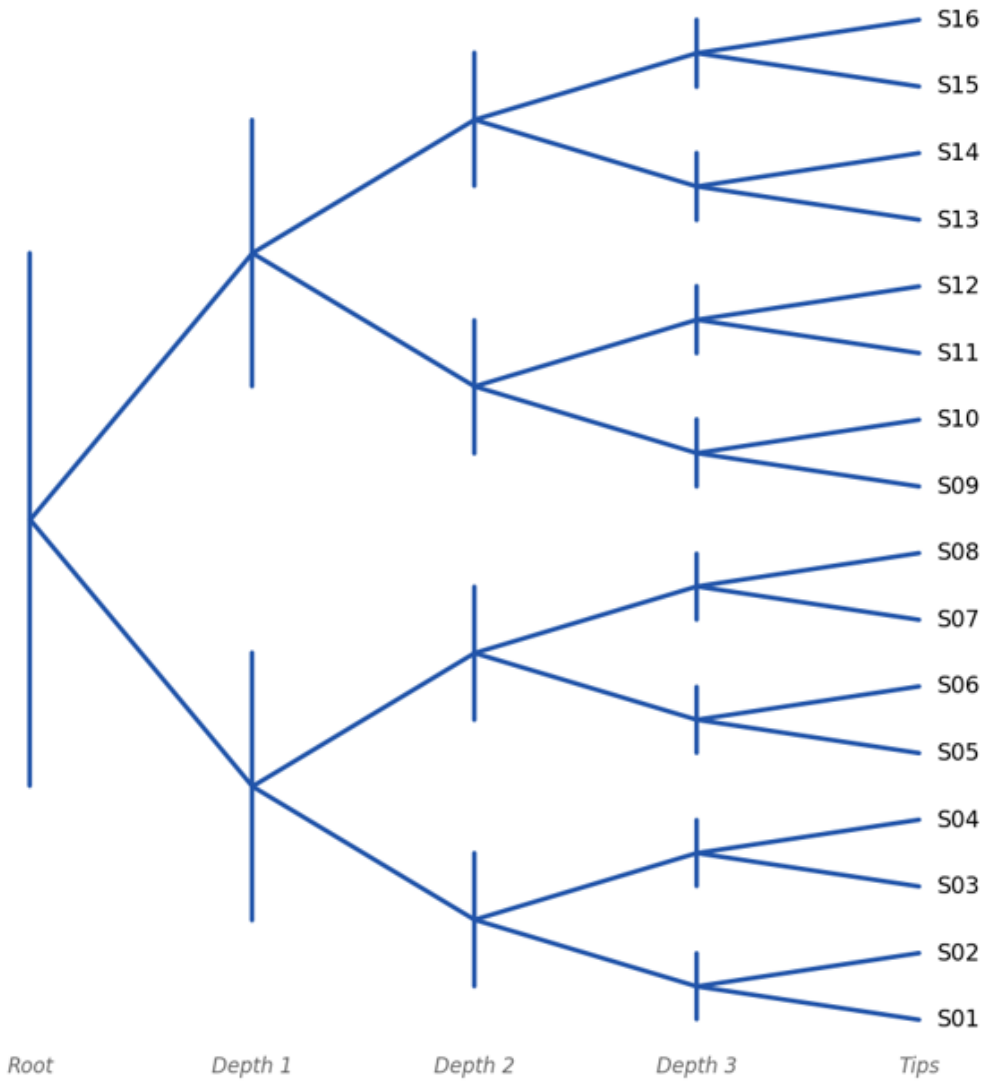
Only local sequence-level mutation parameters were scaled. Point mutations are single-base changes. Small insertions add short pieces of sequence, and small deletions remove short pieces of sequence. Larger rearrangement parameters, including duplication, deletion, translocation, and inversion rates, were left unchanged from the baseline parameter file.

This was an intentional choice. Aevol can model larger genome rearrangements, and those are biologically and computationally interesting. However, changing rearrangement rates at the same time as point mutation and small indel rates would make the first benchmark harder to interpret. By scaling only local mutation parameters, this pilot focused on a cleaner question: what happens to tree recovery when local sequence-level mutation pressure increases?

2.4 True tree generation

A balanced 16-tip species tree was generated and used as the known ground truth for all mutation-rate regimes. The same true tree was used in every condition so that differences in tree recovery could be interpreted relative to mutation-rate treatment rather than differences in tree shape.

**Figure 3: Known true 16-tip balanced species tree (all branch lengths equal; used as ground truth in all conditions)**




The tree contained 16 terminal tips and 15 internal nodes. The tree manifest contained 30 branches or edges, which matches the expected structure for a fully bifurcating 16-tip tree. A bifurcating tree is one in which each internal node splits into two descendant branches. This is a common representation in phylogenetics because it describes a series of branching events.

Using a balanced tree is a limitation, but also a reasonable starting point. It creates a controlled test case where errors are easier to inspect. Once the pipeline is stable across more replicates, future versions could use less balanced trees, more tips, or variable branch lengths to create a harder and more realistic benchmark.

2.5 FASTA extraction, cleaning, and quality control

For each mutation-rate condition, Aevol produced final genomes for all 16 terminal tips in rep_001. Raw final FASTA files were frozen under data/raw_final_fastas/, preserving the direct simulation outputs before downstream processing. Cleaned FASTA files were then generated under data/clean_fastas/, with standardized tip labels from S01 through S16.

This cleaning step was important because phylogenetic tools are sensitive to sequence labels and input consistency. If one method receives a sequence named S01 and another receives the same genome under a longer internal Aevol label, later tree comparisons can become messy or fail entirely. The goal was to make every workflow start from the same standardized set of inputs.

The QC workflow checked that each condition contained exactly 16 records, that FASTA records were valid, that there were no duplicate IDs, that there were no duplicate sequences, and that genome lengths could be summarized. All five mutation-rate conditions passed these integrity checks.

| Condition | Records | Min length | Max length | Mean length | Status |
|---|---|---|---|---|---|
| very_low | 16 | 2869 | 3107 | 2969.19 | ok |
| low | 16 | 2830 | 3113 | 2975.94 | ok |
| normal | 16 | 2798 | 2963 | 2900.62 | ok |
| high | 16 | 1993 | 2585 | 2281.00 | ok |
| very_high | 16 | 640 | 1044 | 813.81 | ok |

The QC results show an important pattern. The three lower mutation-rate conditions had similar mean genome lengths, roughly 2.9 to 3.0 kb. The high condition was shorter, and the very_high condition was much shorter. This matters because tree recovery depends on the amount and quality of comparable sequence information. A high mutation-rate condition may not only produce more substitutions. It may also produce more indel effects, more sequence disruption, and less shared signal for alignment-based or distance-based tools.

2.6 Software environment

The software environment was checked in WSL/Ubuntu and recorded in results/software_versions/phylo_tools.tsv. Major tools included Python, R, SeqKit, Mash, MAFFT, IQ-TREE, progressiveMauve, and optional nucmer/dnadiff. Python packages included Biopython, pandas, and ete3. R packages included ape, phangorn, tidyverse, and dendextend. FastME was not available, so the Mash distance workflow used ape::bionj() in R for tree inference.

The tool choices match the needs of the project. SeqKit is a toolkit for FASTA/Q file manipulation and is useful for sequence-processing tasks. Mash estimates genome distances using MinHash sketches, which makes it useful for an alignment-free comparison workflow. MAFFT is a multiple sequence alignment program for nucleotide or amino-acid sequences. IQ-TREE is a maximum-likelihood phylogenetic inference program used for model-based tree reconstruction. Mauve and progressiveMauve are whole-genome alignment tools designed for cases where genomes may contain rearrangements or other large-scale differences. BioNJ is an improved neighbor-joining-style method that builds a tree from a distance matrix.

2.7 Phylogenetic inference workflows

Three phylogenetic workflows were implemented and tested in rep_001.

2.7.1 Mash/BioNJ

The Mash/BioNJ workflow represented an alignment-free, distance-based strategy. “Alignment-free” means that the method does not first try to line up every base position across all genomes. Instead, Mash estimates how similar or different genomes are based on their sequence content. It does this using MinHash sketches, which are compact summaries of k-mer content. A k-mer is a short substring of length k taken from a sequence. For example, if k = 4, the sequence ATGCG contains the 4-mers ATGC and TGCG.

Mash estimated pairwise genomic distances from the cleaned FASTA inputs. Those distances were converted into a square distance matrix, and ape::bionj() was used to infer a tree. A distance matrix is a table where each cell represents the estimated distance between two genomes. BioNJ then uses that matrix to build a tree that attempts to represent those pairwise distances. Mash is useful for large-scale genome comparison because its sketching approach allows fast distance estimation without full alignment.

This workflow asks a direct question: if the genomes are compared through alignment-free genome distances, does the resulting distance structure still contain enough information to recover the known tree?

2.7.2 MAFFT/IQ-TREE

The MAFFT/IQ-TREE workflow represented an alignment-based maximum-likelihood strategy. A multiple sequence alignment attempts to place homologous positions into the same columns. Homologous positions are sequence positions that are inferred to share common ancestry. Once sequences are aligned, a phylogenetic method can treat each alignment column as evidence about evolutionary relationships.

MAFFT was used to align all 16 cleaned tip genomes for each condition. IQ-TREE then inferred a maximum-likelihood tree from the resulting alignment. Maximum likelihood, in this context, means that the software searches for the tree that best explains the observed aligned sequence data under a model of sequence evolution. IQ-TREE is designed for efficient maximum-likelihood phylogenetic inference and supports model-based analysis of sequence alignments.

This workflow differs from Mash/BioNJ because it preserves site-level information. Instead of reducing each genome pair to one distance estimate, it attempts to infer relationships from patterns across aligned characters. However, MAFFT is not a rearrangement-aware whole-genome aligner. In this project, it should be understood as a useful full-sequence alignment baseline, not as a method specifically designed to model large-scale genome architecture changes.

2.7.3 Mauve/IQ-TREE

The Mauve/IQ-TREE workflow represented a whole-genome alignment strategy. Whole-genome alignment is different from ordinary sequence alignment because it tries to identify corresponding regions across entire genomes, even when those genomes may have rearrangements, segmental changes, or regions that are not shared by all genomes.

The combined FASTA file was split into individual genome FASTA files, progressiveMauve was used to align the genomes, the XMFA output was converted into a strict shared core alignment across all expected tips, and IQ-TREE was run when that core alignment was usable. Mauve was designed to identify and align conserved genomic regions in the presence of rearrangements, and progressiveMauve extends that logic to multiple genomes with possible gene gain, loss, and rearrangement.

The strict shared-core alignment rule was important. This workflow did not simply ask whether progressiveMauve could produce some alignment output. It asked whether enough sequence could be recovered as shared across all 16 tips to support tree inference. At high and very_high, the workflow was tested, but it did not produce scored trees because many tips were represented mostly or entirely by gaps in the recovered core alignment. This should be reported as a workflow limitation under the strict core-alignment rule, not as an untested condition and not as a general claim that progressiveMauve fails on divergent genomes.

2.8 Tree validation and scoring

Inferred trees were compared to the known true tree stored at data/true_trees/true_tree_16tips.nwk. The scoring script checked whether each inferred tree existed, whether it contained the expected 16 tips, whether tip labels matched the true tree, whether branch lengths were valid, and how the inferred tree's splits compared to the true tree's splits.

Robinson-Foulds distance was used as the primary tree-recovery metric. A split, or bipartition, is the division of tips created by cutting one internal branch of a tree. Robinson-Foulds distance compares two trees by counting splits that are present in one tree but absent in the other. A normalized RF distance of 0 indicates perfect topological agreement with the true tree. Higher values indicate more disagreement.

Patristic distance correlations were also calculated as secondary diagnostics. A patristic distance is the distance between two tips measured along the branches of a tree. These values can be useful, but they require caution here because different workflows estimate branch lengths in different ways. A distance-based method, a likelihood-based method, and a core-alignment method may all produce branch lengths with different assumptions and scales. For that reason, RF distance was treated as the primary result because the central benchmark question was whether the correct topology was recovered.

---

## 3. Results

3.1 Simulation and FASTA generation

Aevol successfully produced final genomes for all five mutation-rate regimes in rep_001. Each condition contained 16 final tip genomes labeled S01 through S16 and a combined all_tips.fa file. Raw simulation FASTA outputs were frozen, cleaned FASTA inputs were generated, and all five conditions passed FASTA integrity checks.

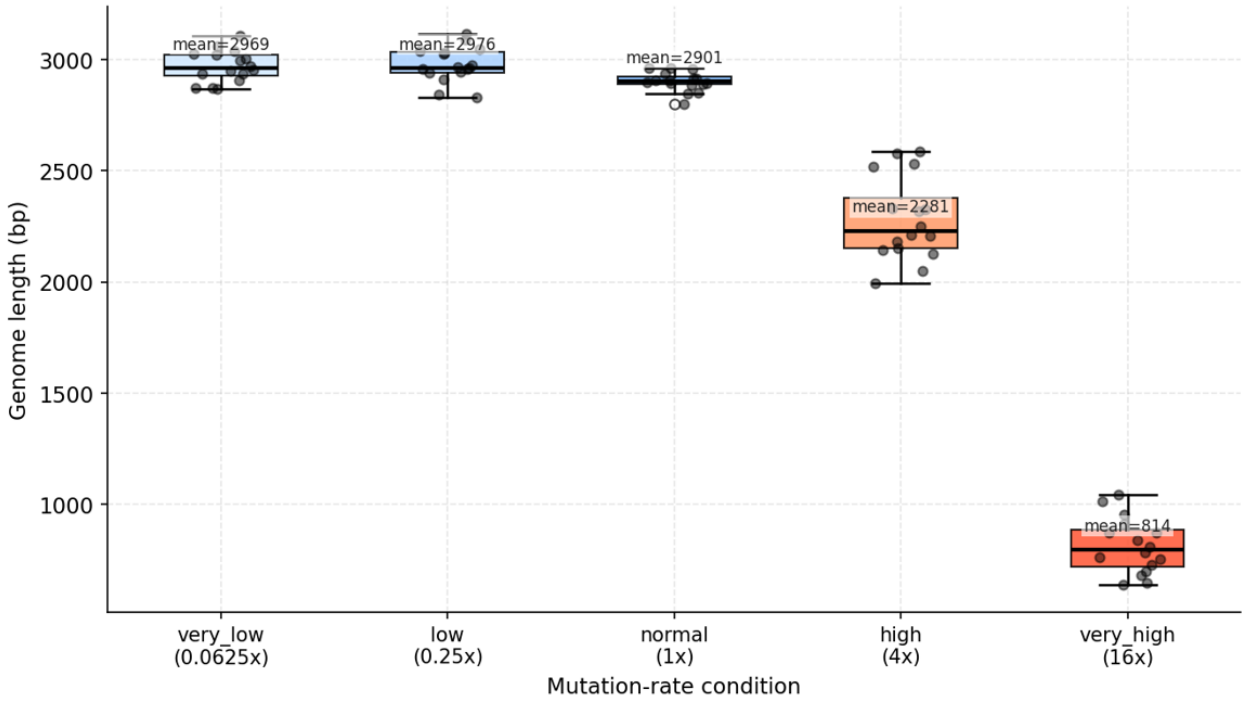
This first result matters because it confirms that the workflow worked from simulation through data preparation. Before comparing phylogenetic methods, it was necessary to verify that every condition had the expected number of tip sequences, that the sequences were valid, and that all methods were receiving comparable inputs.

3.2 Genome length changed across mutation-rate regimes

The clearest QC pattern was the difference in final genome length across mutation-rate treatments. The very_low, low, and normal conditions produced genomes with similar mean

lengths, all close to 3 kb. The high condition showed a substantial decrease in mean genome length, and the very_high condition showed an even stronger reduction.

**Figure 1: Final genome length by mutation-rate condition (rep_001)**

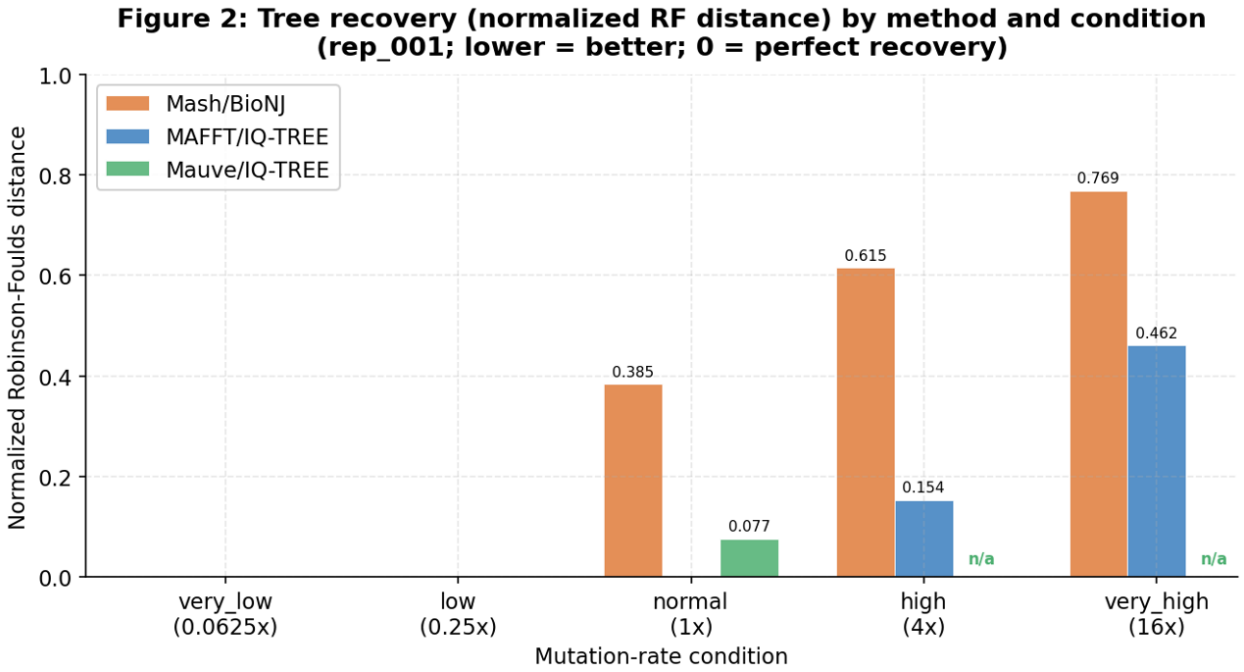



This result suggests that in rep_001, increasing local mutation rate was associated not only with more sequence-level divergence, but also with shorter final genomes. That distinction is important. If genome length stayed constant, a higher mutation rate could mainly be interpreted as a higher density of substitutions and small indels across comparable sequences. Here, especially in the very_high treatment, the genomes became much shorter. That likely reduced the amount of comparable sequence available to downstream methods.

This does not prove a general rule yet because there is only one replicate. Still, it is one of the most important observations from the pilot. Any interpretation of tree-recovery failure at high mutation rates should consider both mutation-driven divergence and the change in genome length. The very_high condition is not simply a longer-distance version of the lower-rate conditions. It appears to be a much more disrupted dataset.

3.3 Tree recovery varied by mutation rate and method

The main phylogenetic result was that tree-reconstruction error increased with mutation rate, but not evenly across methods.




At very_low and low, all successful methods recovered the true topology perfectly. This indicates that, at least in this pilot replicate, the lower mutation-rate regimes preserved enough tree signal for every implemented workflow to recover the known species tree.

At normal, the methods started to separate. MAFFT/IQ-TREE still recovered the true topology perfectly, Mauve/IQ-TREE was very close to the true topology, and Mash/BioNJ showed noticeable topological error. This was the first condition where the workflows clearly behaved differently.

At high, both Mash/BioNJ and MAFFT/IQ-TREE produced scored trees, but both showed error. Mash/BioNJ had much higher normalized RF distance than MAFFT/IQ-TREE. Mauve/IQ-TREE did not produce a scored tree under the strict shared-core rule.

At very_high, the stress became more obvious. Mash/BioNJ had the highest error among the scored methods, MAFFT/IQ-TREE still produced a tree but with substantial error, and Mauve/IQ-TREE again did not produce a scored tree. These results suggest that higher mutation-rate treatments made the known tree harder to recover, but they also show that the form of failure depended on the workflow.

### 3.4 Method-specific outcomes

Mash/BioNJ recovered the true tree at very_low and low, but its normalized RF distance increased from normal onward. The score rose to 0.385 at normal, 0.615 at high, and 0.769 at very_high. This pattern suggests that the alignment-free distance signal became less reliable as mutation pressure increased. One possible explanation is that high divergence

and genome shortening reduced the amount of shared k-mer content among related genomes. Since Mash estimates distances from sequence sketches, its performance depends on genome-wide similarity patterns remaining informative.

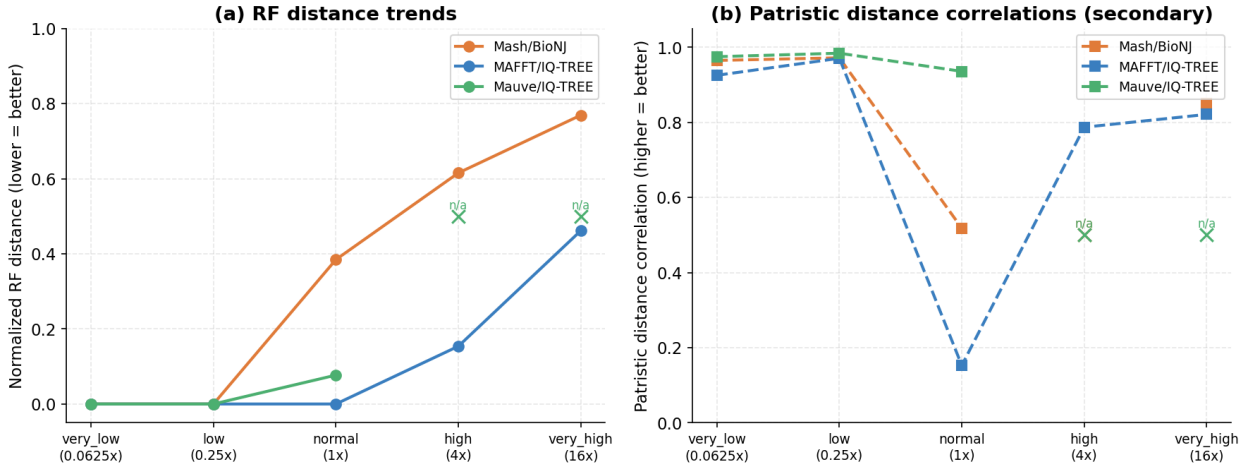
MAFFT/IQ-TREE performed best overall in rep_001. It recovered the true topology at very_low, low, and normal. At high, it showed a normalized RF distance of 0.154, and at very_high, it increased to 0.462. That is still substantial error at the highest mutation rate, but it remained lower than Mash/BioNJ at both high and very_high. One interpretation is that alignment plus maximum-likelihood inference retained useful character-level signal longer than the distance-only workflow. This should be stated carefully, though. MAFFT/IQ-TREE was the strongest method in this pilot replicate, but one replicate is not enough to claim that it is generally superior for Aevol 4b mutation-rate benchmarks.

Mauve/IQ-TREE recovered the true tree at very_low and low, and it remained close to the true topology at normal, with normalized RF distance of 0.077. At high and very_high, however, the workflow did not produce scored trees. This result needs careful wording because "missing tree" can sound like the method was skipped. It was not. The workflow was tested, but the strict shared-core alignment extraction became unusable because many tips were represented mostly by gaps in the recovered core alignment. That is a meaningful benchmark result because it shows that the high-divergence conditions stressed the workflow before tree scoring could even occur.

### 3.5 Patristic correlations as secondary diagnostics

Patristic correlations were calculated, but they were treated as secondary diagnostics rather than primary results.

**Figure 4: RF distance trends and patristic distance correlations by method (rep_001)**




The Mash/BioNJ patristic correlation at high is absent because the inferred tree contained

negative branch lengths, which invalidate patristic distance calculations; this is itself a diagnostically meaningful result indicating distance-based breakdown at that mutation rate. The low MAFFT/IQ-TREE patristic correlation at normal, despite perfect RF recovery, shows why these values need caution. A tree can have the correct topology while still having branch lengths that differ substantially from the true tree. For this pilot, RF distance is the cleaner primary metric because the main question is whether the correct topology was recovered.

---

## 4. Discussion

### 4.1 Main interpretation

The main result from rep_001 is that increasing local sequence-level mutation rate reduced phylogenetic recoverability, but the effect depended strongly on the inference workflow. At very_low and low, all successful workflows recovered the known topology. At normal, Mash/BioNJ began to show topological error while MAFFT/IQ-TREE remained perfect and Mauve/IQ-TREE was nearly perfect. At high and very_high, all workflows were stressed, but they were stressed in different ways.

This is more interesting than simply saying that more mutation makes worse trees. The higher mutation-rate conditions did not only add more differences among genomes. They were also associated with shorter final genomes, especially in the very_high treatment. That means the higher-rate conditions likely reduced recoverable phylogenetic signal in multiple ways at once. There may have been more substitutions, more small indels, less shared sequence, shorter genomes, poorer alignment quality, weaker k-mer overlap, and less reliable core alignment recovery.

The pilot results therefore support the idea that phylogenetic accuracy depends not only on how much evolution has occurred, but also on how each workflow extracts usable signal from the data.

### 4.2 Why the methods behaved differently

The three workflows differ because they “see” the genomes in different ways.

Mash/BioNJ treats the problem as a genome-distance problem. It estimates pairwise distances and then builds a tree from those distances. This is fast and practical, but it depends on the distance matrix preserving enough information about the true branching history. If high mutation pressure reduces shared sequence content or creates noisy distances, the inferred tree can degrade.

MAFFT/IQ-TREE treats the problem as an alignment plus model-based inference problem. MAFFT attempts to align homologous positions across the genomes, and IQ-TREE searches for a topology that explains the aligned sequence data under a model of sequence evolution. This approach kept more useful signal in rep_001, at least compared with Mash/BioNJ. It was not immune to the highest mutation rates, but it degraded more slowly.

Mauve/IQ-TREE treats the problem as a whole-genome alignment and shared-core problem. That makes sense for genome-level comparisons, especially when genome structure may matter. However, the strict shared-core requirement also created a hard failure point. If too little sequence could be recovered as shared across all tips, IQ-TREE could not be meaningfully applied to that core alignment. This explains why Mauve/IQ-TREE performed well through normal, but did not yield scored trees at high and very_high.

The larger takeaway is that method failure is not one thing. A distance method can fail by returning an increasingly inaccurate tree. An alignment-likelihood method can fail more gradually by accumulating topological error. A whole-genome core-alignment method can fail by becoming unable to produce a usable shared alignment under the rules of the workflow. These are all valid benchmark outcomes.

### 4.3 Relationship to Aevol 4b

This project uses the Aevol 4b paper as a conceptual foundation. That work showed that Aevol 4b can generate artificial-life sequence data that are compatible with standard bioinformatics analysis, including phylogenetic reconstruction. This project takes that basic idea and applies it to a focused mutation-rate benchmark with a smaller controlled tree, five local mutation-rate regimes, and a reusable analysis pipeline.

The purpose here is slightly different. The Aevol 4b paper was mainly about demonstrating that Aevol 4b can bridge artificial life and bioinformatics. This project is more about using Aevol 4b as a test system to explore how mutation-rate changes affect tree recovery, while also demonstrating practical bioinformatics skills: simulation design, scripted data handling, tool integration, quality control, phylogenetic inference, and quantitative tree comparison.

### 4.4 Limitations

The most important limitation is that this is a pilot benchmark with only one completed replicate. The results are useful for demonstrating the pipeline and identifying patterns, but they are not enough to make strong statistical claims about method performance. Additional replicates are needed to estimate average behavior, variation among runs, and the consistency of method rankings.

A second limitation is genome length. The final genomes in this project are short, especially in the very_high treatment. Short genomes may make phylogenetic inference more sensitive to stochastic effects, loss of signal, and alignment artifacts. This is especially relevant because the highest mutation-rate condition also had the shortest genomes.

A third limitation is the balanced 16-tip tree. This is useful for a first benchmark because it is simple and controlled, but real evolutionary histories are often uneven. Future versions could test more complex tree shapes, unequal branch lengths, or larger numbers of tips.

A fourth limitation is that only local mutation rates were scaled. This was a good decision for the first version because it made the experiment easier to interpret. However, Aevol can also model larger rearrangements, and those may be highly relevant for whole-genome alignment workflows. This pilot should therefore be described as a local mutation-rate benchmark, not a complete test of all Aevol mutational processes.

A fifth limitation concerns the Mauve/IQ-TREE workflow. The high-divergence failures partly reflect the strict shared-core alignment rule used in this implementation. A less strict core extraction rule, a different missing-data threshold, or an alternate whole-genome alignment strategy might produce scored trees under high and very_high. That would be worth testing before making broader claims about whole-genome alignment performance.

Finally, branch-length-based diagnostics should remain secondary. Patristic correlations and related metrics are informative, but branch lengths are not directly comparable across all methods and scoring contexts in this pilot. RF distance is the more appropriate primary metric for the current question because it focuses on topological recovery.

### 4.5 Future work

The next step is to add rep_002 and rep_003 using the same design. With three replicates, the project could report mean normalized RF distance by method and condition, variation across replicates, and method failure rate. Failure rate should be treated as a real outcome, not just missing data, because one of the important findings from rep_001 is that the Mauve/IQ-TREE workflow became unable to produce scored trees under the strict shared-core rule at high mutation rates.

The next version should also add more diagnostics before and after alignment. Useful additions would include alignment length, percent gaps, number of variable sites, number of parsimony-informative sites, missing-data fraction, and core-alignment length. These values would make it easier to connect tree-recovery error to the underlying sequence data. For example, if normalized RF distance increases at the same time that core-alignment length drops, that would support the idea that loss of shared comparable sequence is contributing to failure.

A future version could also test an alternate whole-genome alignment strategy or a relaxed Mauve core-alignment rule. That would help distinguish between a true lack of recoverable signal and a pipeline-specific threshold that was too strict for high-divergence genomes.

Finally, the report could be strengthened by adding visual comparisons of inferred trees to the true tree. Tanglegrams or split-level summaries would make it easier to see whether errors are shallow tip rearrangements or deeper clade-level mistakes. That would make the project more biologically interpretable than RF distance alone.

---

## 5. Conclusion

This pilot benchmark shows that Aevol 4b can be used to generate controlled, bioinformatics-compatible simulated genomes for testing phylogenomic inference workflows. Using one shared ancestor, one balanced 16-tip species tree, and five local mutation-rate regimes, the project successfully generated final FASTA files, cleaned and validated the sequence inputs, inferred trees with three different workflows, and scored those trees against the known simulated topology.

In rep_001, tree recovery was perfect for all successful workflows at very_low and low mutation rates. At higher mutation rates, accuracy declined in a method-dependent way. Mash/BioNJ degraded from normal upward, MAFFT/IQ-TREE performed best overall in the pilot replicate, and Mauve/IQ-TREE became limited by strict shared-core alignment recovery at high and very_high. The highest mutation-rate conditions were also associated with shorter final genomes, which likely reduced the amount of comparable sequence available for tree inference.

These results should be interpreted as a successful pilot rather than a final statistical benchmark. Even at this stage, though, the project demonstrates a complete and reproducible bioinformatics workflow: simulation design, FASTA processing, software integration, phylogenetic inference, tree scoring, and interpretation against a known evolutionary history. That makes the project useful both as an evolutionary methods exercise and as a demonstration of practical coding and data-management skills.

---

## References

1. Daudey H, Parsons DP, Tannier E, Daubin V, Boussau B, Liard V, Rouzaud-Cornabas J, Beslon G. Aevol 4b: Bridging the gap between artificial life and bioinformatics. In:

*Proceedings of the 2024 Conference on Artificial Life (ALIFE 2024)*. 2024;36:6. doi:10.1162/isal_a_00716.

2. Robinson DF, Foulds LR. Comparison of phylogenetic trees. *Math Biosci*. 1981;53(1–2):131–147. doi:10.1016/0025-5564(81)90043-2.

3. Ondov BD, Treangen TJ, Melsted P, Mallonee AB, Bergman NH, Koren S, Phillippy AM. Mash: fast genome and metagenome distance estimation using MinHash. *Genome Biol*. 2016;17(1):132. doi:10.1186/s13059-016-0997-x.

4. Gascuel O. BIONJ: an improved version of the NJ algorithm based on a simple model of sequence data. *Mol Biol Evol*. 1997;14(7):685–695.

5. Paradis E, Schliep K. ape 5.0: an environment for modern phylogenetics and evolutionary analyses in R. *Bioinformatics*. 2019;35(3):526–528. doi:10.1093/bioinformatics/bty633.

6. Katoh K, Standley DM. MAFFT Multiple Sequence Alignment Software Version 7: Improvements in performance and usability. *Mol Biol Evol*. 2013;30(4):772–780. doi:10.1093/molbev/mst010.

7. Nguyen LT, Schmidt HA, von Haeseler A, Minh BQ. IQ-TREE: A fast and effective stochastic algorithm for estimating maximum-likelihood phylogenies. *Mol Biol Evol*. 2015;32(1):268–274. doi:10.1093/molbev/msu300.

8. Darling AE, Mau B, Perna NT. progressiveMauve: Multiple genome alignment with gene gain, loss and rearrangement. *PLOS ONE*. 2010;5(6):e11147. doi:10.1371/journal.pone.0011147.

9. Shen W, Le S, Li Y, Hu F. SeqKit: A cross-platform and ultrafast toolkit for FASTA/Q file manipulation. *PLOS ONE*. 2016;11(10):e0163962. doi:10.1371/journal.pone.0163962.

10. Schliep KP. phangorn: Phylogenetic analysis in R. *Bioinformatics*. 2011;27(4):592–593. doi:10.1093/bioinformatics/btq706.

11. Cock PJA, Antao T, Chang JT, Chapman BA, Cox CJ, Dalke A, Friedberg I, Hamelryck T, Kauff F, Wilczynski B, de Hoon MJL. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*. 2009;25(11):1422–1423. doi:10.1093/bioinformatics/btp163.